Cheat Sheet: Advanced Flutter

Calling APIs in Flutter

Type	Description	Code Example
Making Your First API Call	Learn how to perform a basic HTTP GET request	

```html
import 'package:http/http.dart' as http;
Future<void> fetchData() async {
  var response =
      await http.get(Uri.parse('https://api.example.com/data'));

  if (response.statusCode == 200) {
    print(response.body);
  } else {
    print('Error: ${response.statusCode}');
  }
}
```

Handling API Errors	Implement error handling for API requests	

```html
import 'package:http/http.dart' as http;

Future<void> fetchData() async {
  try {
    var response =
        await http.get(Uri.parse('https://api.example.com/data'));

    if (response.statusCode == 200) {
      print(response.body);
    } else {
      print('Error: ${response.statusCode}');
    }
  } catch (e) {
    print('Error: $e');
  }
}
```

Parsing JSON Data	Decode JSON data received from an API into Dart objects	

```html
import 'dart:convert';

var response = '{"username": "John"}'; // Example JSON string
var data = jsonDecode(response);

print(data['username']);
```

Asynchronous Programming	Use async and await for asynchronous API calls

```html
import 'package:http/http.dart' as http;
import 'dart:convert';

Future<void> getUserData() async {
  final response = await http.get(
    Uri.parse('https://api.example.com/users/1'),
  );

  if (response.statusCode == 200) {
    final data = jsonDecode(response.body);
    print('User name: ${data['name']}');
  } else {
    print('Failed to fetch user');
  }
}
```

Post Data to API

Send data to an API using the POST method

```html
import 'package:http/http.dart' as http;
import 'dart:convert';

Future<void> postData() async {
  var response = await http.post(
    Uri.parse('https://api.example.com/add'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'name': 'John Doe'}),
  );

  print('Response status: ${response.statusCode}');
}
```

Fetch Data with Headers

Make HTTP requests with additional headers

```html
import 'package:http/http.dart' as http;
Future<void> fetchWithHeaders() async {
  var response = await http.get(
    Uri.parse('https://api.example.com/data'),
    headers: {'Authorization': 'Bearer your_token_here'}
  );
  print('Data with headers: ${response.body}');
}
```

Introduction to mobile native features in Flutter

Type	Description	Code Example
Accessing the Camera	Utilize the native camera functionality via the Flutter plugin	

```html
import 'package:camera/camera.dart';
Future<void> takePicture() async {
  final cameras = await availableCameras();
  final firstCamera = cameras.first;
  final CameraController controller = CameraController(firstCamera, ResolutionPreset.high);
  await controller.initialize();
  final image = await controller.takePicture();
  await controller.dispose();
}
```

Using GPS

Fetch the current location using the geolocation plugin

html

```
import 'package:geolocator/geolocator.dart';
bool serviceEnabled = await Geolocator.isLocationServiceEnabled();
if (!serviceEnabled) {
  print('Location services are disabled');
  return;
}
LocationPermission permission = await Geolocator.requestPermission();

if (permission == LocationPermission.denied ||
    permission == LocationPermission.deniedForever) {
  print('Location permission denied');
  return;
}

Position position = await Geolocator.getCurrentPosition();
print(position.latitude);
```

Local Storage

Store data locally using the shared_preferences plugin

html

```
import 'package:shared_preferences/shared_preferences.dart';
final prefs = await SharedPreferences.getInstance();
await prefs.setString('username', 'exampleUser');
```

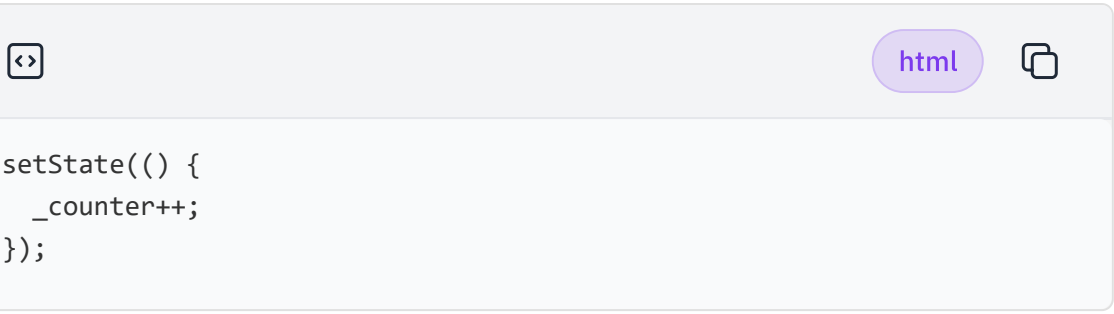
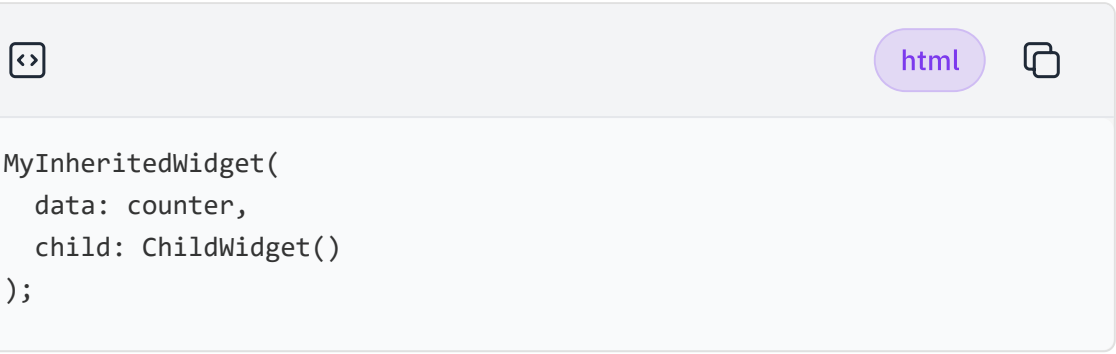
Accessing Device Sensors	Interact with native device sensors like the accelerometer

```html
import 'dart:async';
import 'package:sensors_plus/sensors_plus.dart';
StreamSubscription subscription =
    accelerometerEvents.listen((event) {
  print(event);
});

// Later when done
subscription.cancel();
```

Managing Notifications	Schedule and manage notifications locally on the device

```html
import 'package:flutter_local_notifications/flutter_local_notifications.dart';
final FlutterLocalNotificationsPlugin notificationsPlugin =
    FlutterLocalNotificationsPlugin();

const androidSettings = AndroidInitializationSettings('@mipmap/ic_launcher');
const initSettings = InitializationSettings(android: androidSettings);

await notificationsPlugin.initialize(initSettings);

const androidDetails = AndroidNotificationDetails(
  'channelId',
  'channelName',
  channelDescription: 'channelDescription',
);

const notificationDetails = NotificationDetails(android: androidDetails);

await notificationsPlugin.show(
  0,
  'Test Title',
  'Test Body',
  notificationDetails,
);
```

Managing state in Flutter

Type	Description	Code Example
Manage local widget state changes in a StatefulWidget	Provides a way to manage and update the local state of a widget	
InheritedWidget	Used to pass data down the widget tree (advanced concept)	

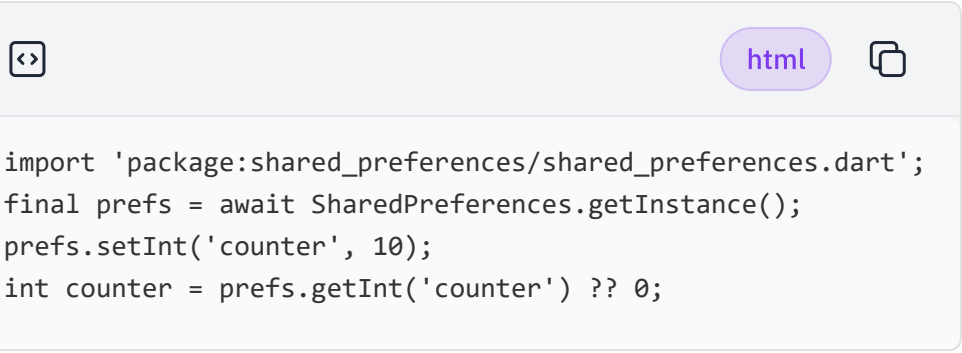
GlobalKey and FormState	Utilize GlobalKey to access the state from anywhere in the app, commonly used with forms	

```html
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: Column(
    children: [
      TextFormField(
        validator: (value) =>
            value == null || value.isEmpty ? 'Cannot be empty' : null,
      ),
      ElevatedButton(
        onPressed: () {
          if (_formKey.currentState!.validate()) {
            // Process data
          }
        },
        child: Text('Submit'),
      )
    ],
  ),
);
```

Provider for state management	A wrapper around InheritedWidget to make state management easier and more efficient

```html
import 'package:provider/provider.dart';
class DataModel extends ChangeNotifier {
  int value = 0;
}
ChangeNotifierProvider(
  create: (context) => DataModel(),
  child: Consumer<DataModel>(
    builder: (context, dataModel, child) {
      return Text('${dataModel.value}');
    },
  ),
);
```

Flutter Persistence with Local Storage

Type	Description	Code Example
SharedPreferences	Ideal for simple data such as user preferences and settings	
SQLite	Ideal for structured data and complex queries. It works like a traditional database	
Files	Read/write files for documents or media	
Third-Party Packages	**Hive**: A fast, NoSQL database for Flutter. **LocalStorage**: A third-party package for storing JSON-like data locally.	
Initialize Local Storage	Set up the chosen storage method at the start of your app.	

CRUD Operations	Implement Create, Read, Update, and Delete functions.	 html 

```
// Saving data
prefs.setInt('counter', 10);
// Retrieving data
int counter = prefs.getInt('counter') ?? 0;
// Deleting data
prefs.remove('counter');
```

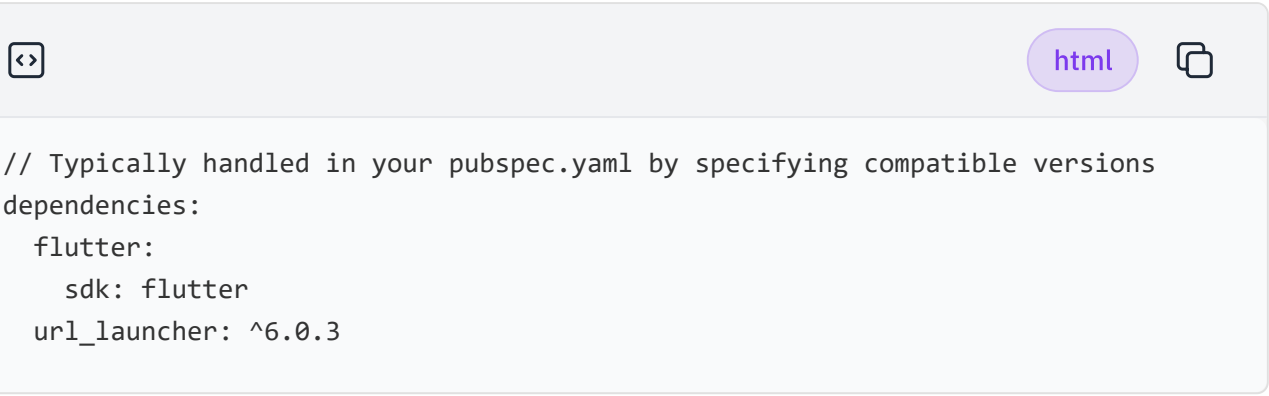
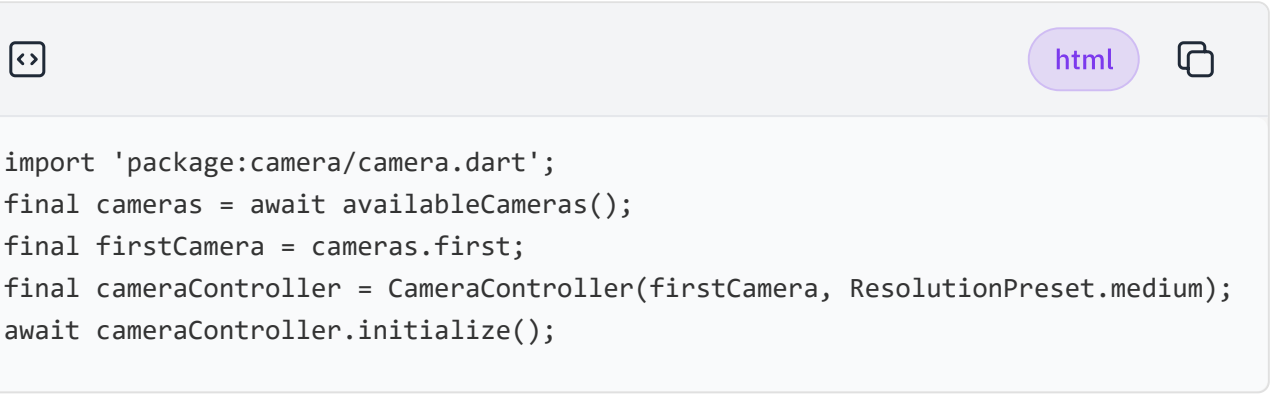
Serialization/Deserialization	Encode and decode data using JSON for complex structures	 html 

```
import 'dart:convert';
class User {
  final String name;

 User({required this.name});

  factory User.fromJson(Map<String, dynamic> json) {
    return User(name: json['name']);
  }
}

String jsonString = '{"name": "John"}';

Map<String, dynamic> userMap = jsonDecode(jsonString);
var user = User.fromJson(userMap);
```

Integrating with State Management	Use state management tools (e.g., Provider, Riverpod) to handle local storage updates dynamically in the UI

```html
class UserProvider with ChangeNotifier {
  User? _user;

  User? get user => _user;

  Future<void> loadUser() async {
    final prefs = await SharedPreferences.getInstance();
    String? userJson = prefs.getString('user');

    if (userJson != null) {
      _user = User.fromJson(jsonDecode(userJson));
      notifyListeners();
    }
  }
}
```

Plugins in Flutter

Type	Description	Code Example
Using the url_launcher Plugin	Open URLs using the latest launchUrl API	
Managing Plugin Compatibility	Ensure that plugins are compatible with the version of Flutter you are using	
Camera Plugin	Access the device camera to capture photos and videos	

Audio Players Plugin	Play audio files and streams in Flutter with the Audio Players plugin

```html
import 'package:audioplayers/audioplayers.dart';
final player = AudioPlayer();
await player.play(UrlSource('https://example.com/audio.mp3'));
```

Flutter Maps Plugin	Integrate maps into your Flutter applications using the maps plugin

```html
import 'package:google_maps_flutter/google_maps_flutter.dart';
void _onMapCreated(GoogleMapController controller) {}
GoogleMap(
  onMapCreated: _onMapCreated,
  initialCameraPosition: CameraPosition(
    target: LatLng(0.0, 0.0),
    zoom: 10.0,
  ),
);
```

Cheat Sheet: Advanced Flutter

Calling APIs in Flutter

Type	Description	Code Example
Making Your First API Call	Learn how to perform a basic HTTP GET request	

```html
import 'package:http/http.dart' as http;
Future<void> fetchData() async {
  var response =
      await http.get(Uri.parse('https://api.example.com/data'));

  if (response.statusCode == 200) {
    print(response.body);
  } else {
    print('Error: ${response.statusCode}');
  }
}
```

Handling API Errors

Implement error handling for API requests

```html
import 'package:http/http.dart' as http;

Future<void> fetchData() async {
  try {
    var response =
        await http.get(Uri.parse('https://api.example.com/data'));

    if (response.statusCode == 200) {
      print(response.body);
    } else {
      print('Error: ${response.statusCode}');
    }
  } catch (e) {
    print('Error: $e');
  }
}
```

Parsing JSON Data

Decode JSON data received from an API into Dart objects

```html
import 'dart:convert';

var response = '{"username": "John"}'; // Example JSON string
var data = jsonDecode(response);

print(data['username']);
```

Asynchronous Programming	Use async and await for asynchronous API calls

```html
import 'package:http/http.dart' as http;
import 'dart:convert';

Future<void> getUserData() async {
  final response = await http.get(
    Uri.parse('https://api.example.com/users/1'),
  );

  if (response.statusCode == 200) {
    final data = jsonDecode(response.body);
    print('User name: ${data['name']}');
  } else {
    print('Failed to fetch user');
  }
}
```

Post Data to API

Send data to an API using the POST method

```html
import 'package:http/http.dart' as http;
import 'dart:convert';

Future<void> postData() async {
  var response = await http.post(
    Uri.parse('https://api.example.com/add'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'name': 'John Doe'}),
  );

  print('Response status: ${response.statusCode}');
}
```

Fetch Data with Headers

Make HTTP requests with additional headers

```html
import 'package:http/http.dart' as http;
Future<void> fetchWithHeaders() async {
  var response = await http.get(
    Uri.parse('https://api.example.com/data'),
    headers: {'Authorization': 'Bearer your_token_here'}
  );
  print('Data with headers: ${response.body}');
}
```

Introduction to mobile native features in Flutter

Type	Description	Code Example
Accessing the Camera	Utilize the native camera functionality via the Flutter plugin	

```html
import 'package:camera/camera.dart';
Future<void> takePicture() async {
  final cameras = await availableCameras();
  final firstCamera = cameras.first;
  final CameraController controller = CameraController(firstCamera, ResolutionPreset.high);
  await controller.initialize();
  final image = await controller.takePicture();
  await controller.dispose();
}
```

Using GPS

Fetch the current location using the geolocation plugin

```html
import 'package:geolocator/geolocator.dart';
bool serviceEnabled = await Geolocator.isLocationServiceEnabled();
if (!serviceEnabled) {
  print('Location services are disabled');
  return;
}
LocationPermission permission = await Geolocator.requestPermission();

if (permission == LocationPermission.denied ||
    permission == LocationPermission.deniedForever) {
  print('Location permission denied');
  return;
}

Position position = await Geolocator.getCurrentPosition();
print(position.latitude);
```

Local Storage

Store data locally using the shared_preferences plugin

```html
import 'package:shared_preferences/shared_preferences.dart';
final prefs = await SharedPreferences.getInstance();
await prefs.setString('username', 'exampleUser');
```

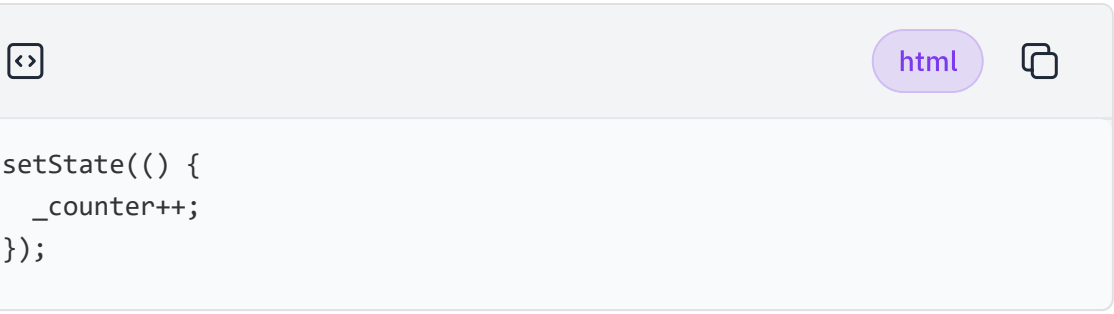
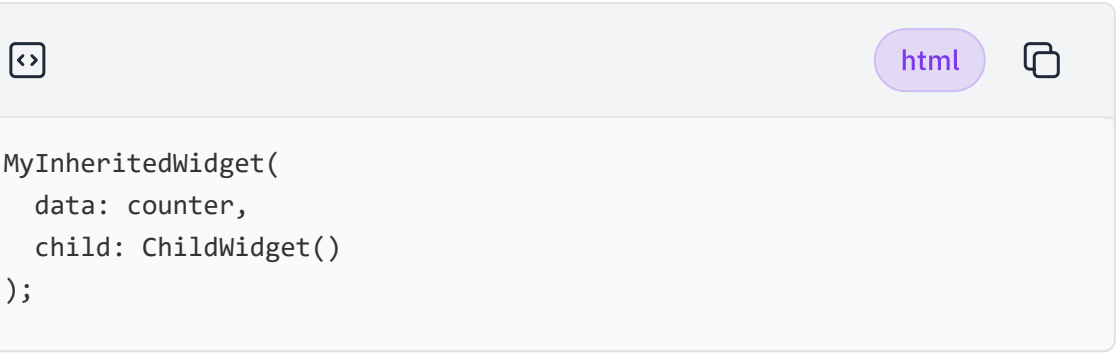
Accessing Device Sensors	Interact with native device sensors like the accelerometer

```html
import 'dart:async';
import 'package:sensors_plus/sensors_plus.dart';
StreamSubscription subscription =
    accelerometerEvents.listen((event) {
  print(event);
});

// Later when done
subscription.cancel();
```

Managing Notifications	Schedule and manage notifications locally on the device

```html
import 'package:flutter_local_notifications/flutter_local_notifications.dart';
final FlutterLocalNotificationsPlugin notificationsPlugin =
    FlutterLocalNotificationsPlugin();

const androidSettings = AndroidInitializationSettings('@mipmap/ic_launcher');
const initSettings = InitializationSettings(android: androidSettings);

await notificationsPlugin.initialize(initSettings);

const androidDetails = AndroidNotificationDetails(
  'channelId',
  'channelName',
  channelDescription: 'channelDescription',
);

const notificationDetails = NotificationDetails(android: androidDetails);

await notificationsPlugin.show(
  0,
  'Test Title',
  'Test Body',
  notificationDetails,
);
```

Managing state in Flutter

Type	Description	Code Example
Manage local widget state changes in a StatefulWidget	Provides a way to manage and update the local state of a widget	
InheritedWidget	Used to pass data down the widget tree (advanced concept)	

```
setState(() {
  _counter++;
});
```

```
MyInheritedWidget(
  data: counter,
  child: ChildWidget()
);
```

GlobalKey and FormState	Utilize GlobalKey to access the state from anywhere in the app, commonly used with forms	

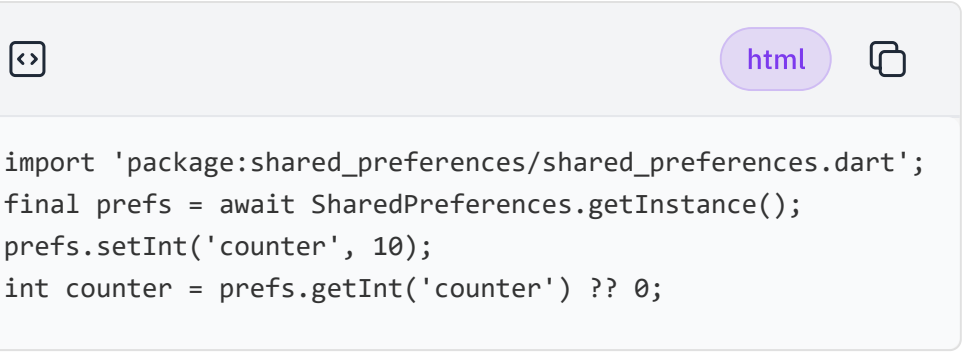
html

```
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: Column(
    children: [
      TextFormField(
        validator: (value) =>
            value == null || value.isEmpty ? 'Cannot be empty' : null,
      ),
      ElevatedButton(
        onPressed: () {
          if (_formKey.currentState!.validate()) {
            // Process data
          }
        },
        child: Text('Submit'),
      )
    ],
  ),
);
```

Provider for state management	A wrapper around InheritedWidget to make state management easier and more efficient

```html
import 'package:provider/provider.dart';
class DataModel extends ChangeNotifier {
  int value = 0;
}
ChangeNotifierProvider(
  create: (context) => DataModel(),
  child: Consumer<DataModel>(
    builder: (context, dataModel, child) {
      return Text('${dataModel.value}');
    },
  ),
);
```

Flutter Persistence with Local Storage

Type	Description	Code Example
SharedPreferences	Ideal for simple data such as user preferences and settings	
SQLite	Ideal for structured data and complex queries. It works like a traditional database	
Files	Read/write files for documents or media	
Third-Party Packages	**Hive**: A fast, NoSQL database for Flutter. **LocalStorage**: A third-party package for storing JSON-like data locally.	
Initialize Local Storage	Set up the chosen storage method at the start of your app.	

CRUD Operations	Implement Create, Read, Update, and Delete functions.	 html 

```
// Saving data
prefs.setInt('counter', 10);
// Retrieving data
int counter = prefs.getInt('counter') ?? 0;
// Deleting data
prefs.remove('counter');
```

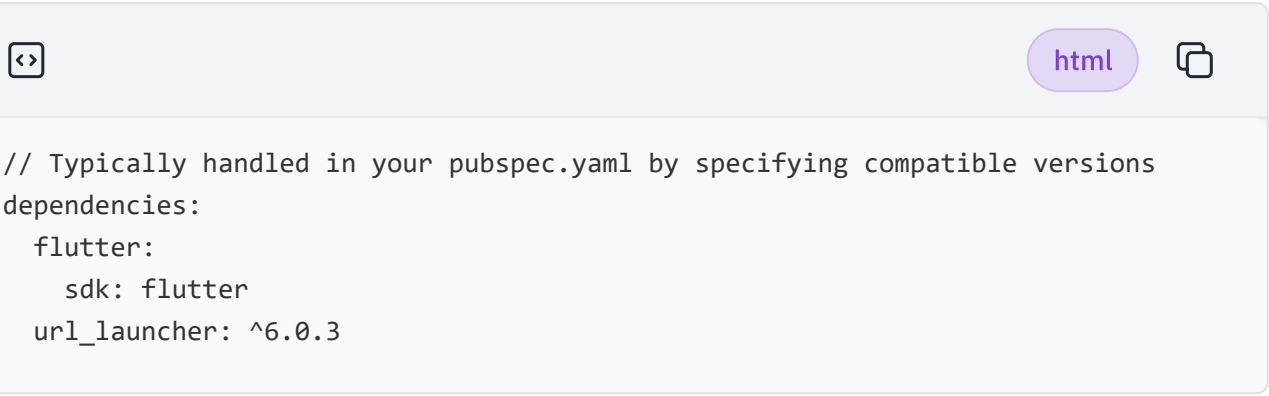
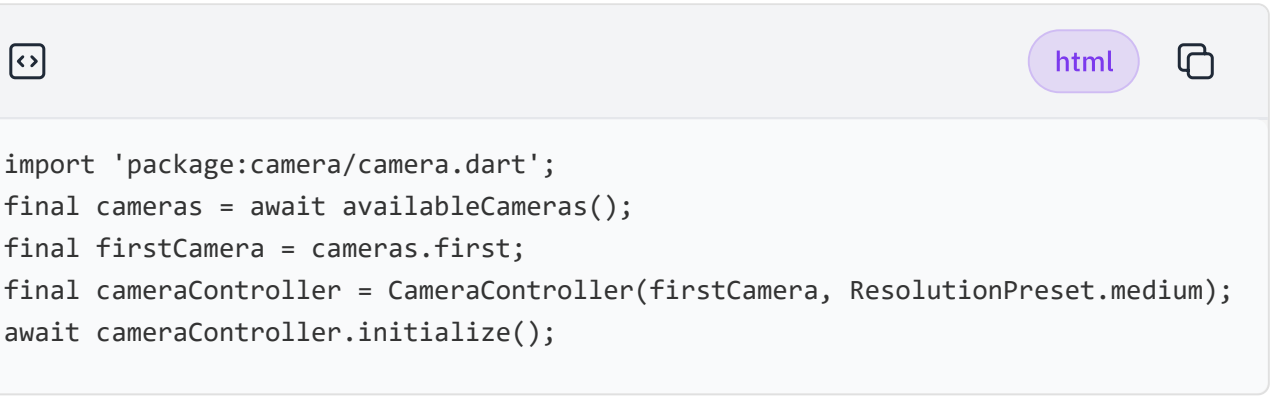
Serialization/Deserialization	Encode and decode data using JSON for complex structures	 html 

```
import 'dart:convert';
class User {
  final String name;

 User({required this.name});

  factory User.fromJson(Map<String, dynamic> json) {
    return User(name: json['name']);
  }
}

String jsonString = '{"name": "John"}';

Map<String, dynamic> userMap = jsonDecode(jsonString);
var user = User.fromJson(userMap);
```

Integrating with State Management	Use state management tools (e.g., Provider, Riverpod) to handle local storage updates dynamically in the UI	

```html
class UserProvider with ChangeNotifier {
  User? _user;

  User? get user => _user;

  Future<void> loadUser() async {
    final prefs = await SharedPreferences.getInstance();
    String? userJson = prefs.getString('user');

    if (userJson != null) {
      _user = User.fromJson(jsonDecode(userJson));
      notifyListeners();
    }
  }
}
```

Plugins in Flutter

Type	Description	Code Example
Using the url_launcher Plugin	Open URLs using the latest launchUrl API	
Managing Plugin Compatibility	Ensure that plugins are compatible with the version of Flutter you are using	
Camera Plugin	Access the device camera to capture photos and videos	

Audio Players Plugin	Play audio files and streams in Flutter with the Audio Players plugin	

```html
import 'package:audioplayers/audioplayers.dart';
final player = AudioPlayer();
await player.play(UrlSource('https://example.com/audio.mp3'));
```

Flutter Maps Plugin	Integrate maps into your Flutter applications using the maps plugin	

```html
import 'package:google_maps_flutter/google_maps_flutter.dart';
void _onMapCreated(GoogleMapController controller) {}
GoogleMap(
  onMapCreated: _onMapCreated,
  initialCameraPosition: CameraPosition(
    target: LatLng(0.0, 0.0),
    zoom: 10.0,
  ),
);
```